

SIMATS ENGINEERING

Department of Computer Science and Engineering ASSESSMENT TOOL2 - CONCEPT MAPPING

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO1	Assessment:	ASSESSMENT TOOL 2- CONCEPT MAPPING
Weightage:	35%	Total Marks:	25
CO1 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)		
Student Name:		Reg.No.:	
Department:		Date:	

Instruction to Students

1. Answer two questions as per the Instruction.
2. Each question carries 12.5 mark.
3. Only the appropriate Tools can be used
4. Time allotted for the exam is 60 min

ANNEXURE A

1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.
2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.

3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.

4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.

5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.

Code of Conduct:

I M.HARINI(192424011) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.Harini

MARKS ALLOCATION

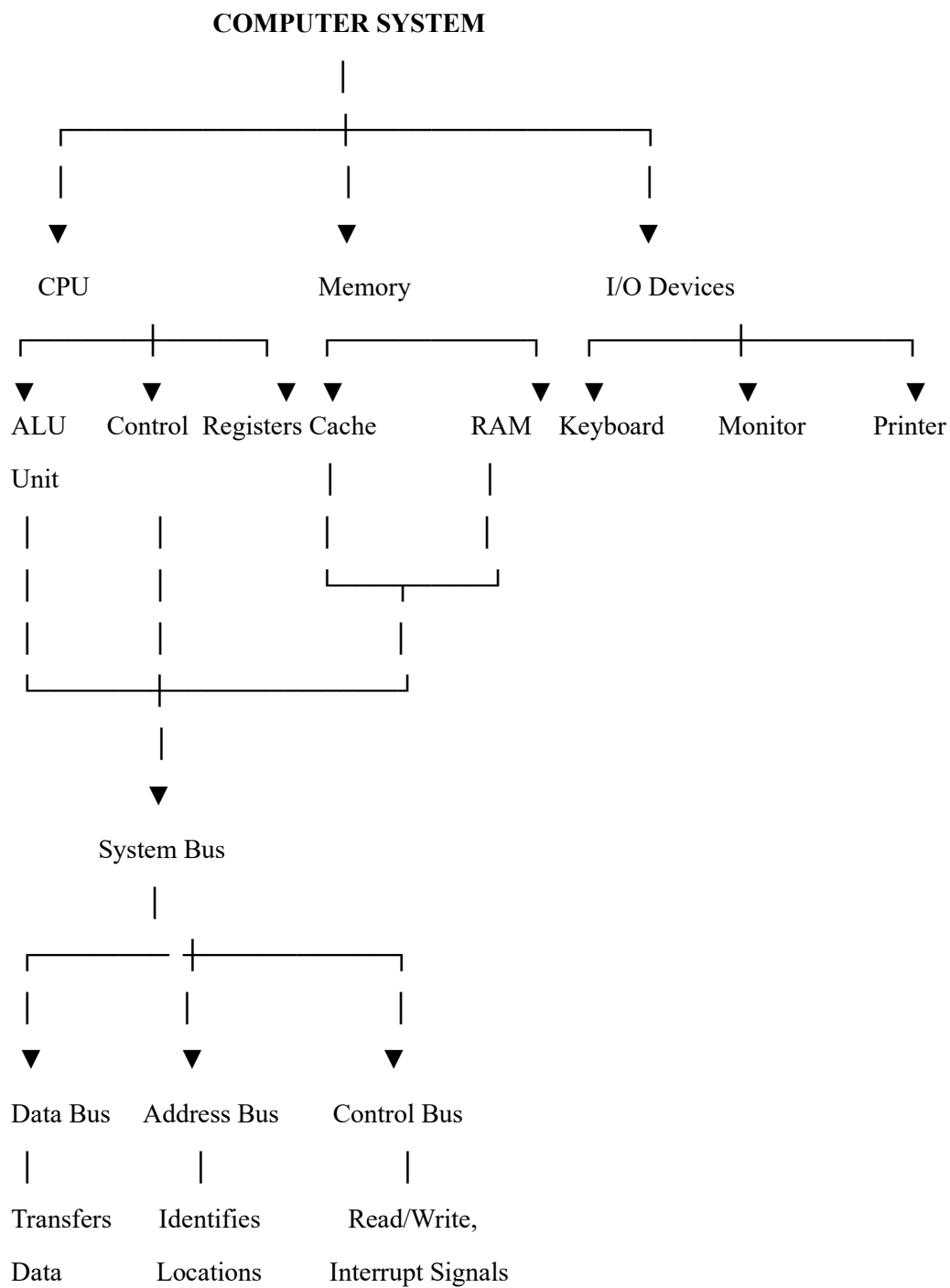
Criteria	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)
Q1					
Q2					

RUBRICS FOR CALCULATION:

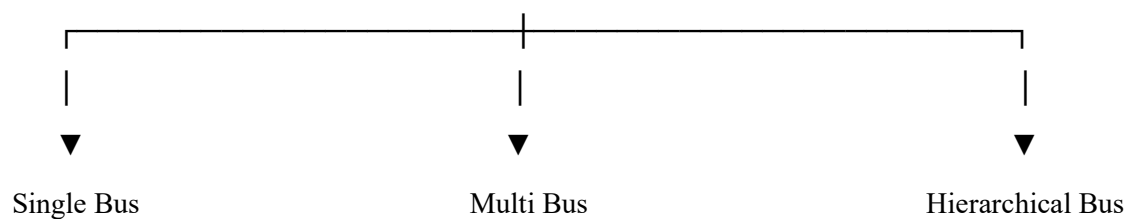
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts

Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

1. Concept Map: CPU, Memory, I/O Devices and Bus Structures



BUS STRUCTURES



|
One common bus
for CPU, RAM,
and I/O

|
▼
Simple Design
Low Cost
More Delay

|
Separate buses
for CPU, Memory
and I/O

|
▼
Parallel Transfer
Less Contention
Higher Speed

|
Multiple levels
Local + System Bus

|
▼
Faster Communication
Better Scalability
Highest Performance

DATA FLOW DURING EXECUTION

Program Stored in RAM

|
▼
Cache checks availability

|
Cache Hit

|
▼
CPU Fetches Instruction

|
▼
Control Unit Decodes

|
▼
Registers Hold Operands

|
▼
ALU Executes Operation

|
▼

Result Stored in Registers



Written Back to RAM or Sent to I/O

POSSIBLE BOTTLENECKS

CPU Waiting for RAM



Memory Latency

Many Devices Sharing Same Bus



Bus Contention

Slow I/O Devices



I/O Bottleneck

PERFORMANCE IMPROVEMENTS

Cache Memory



Reduces Memory Access Time

Multi-Bus



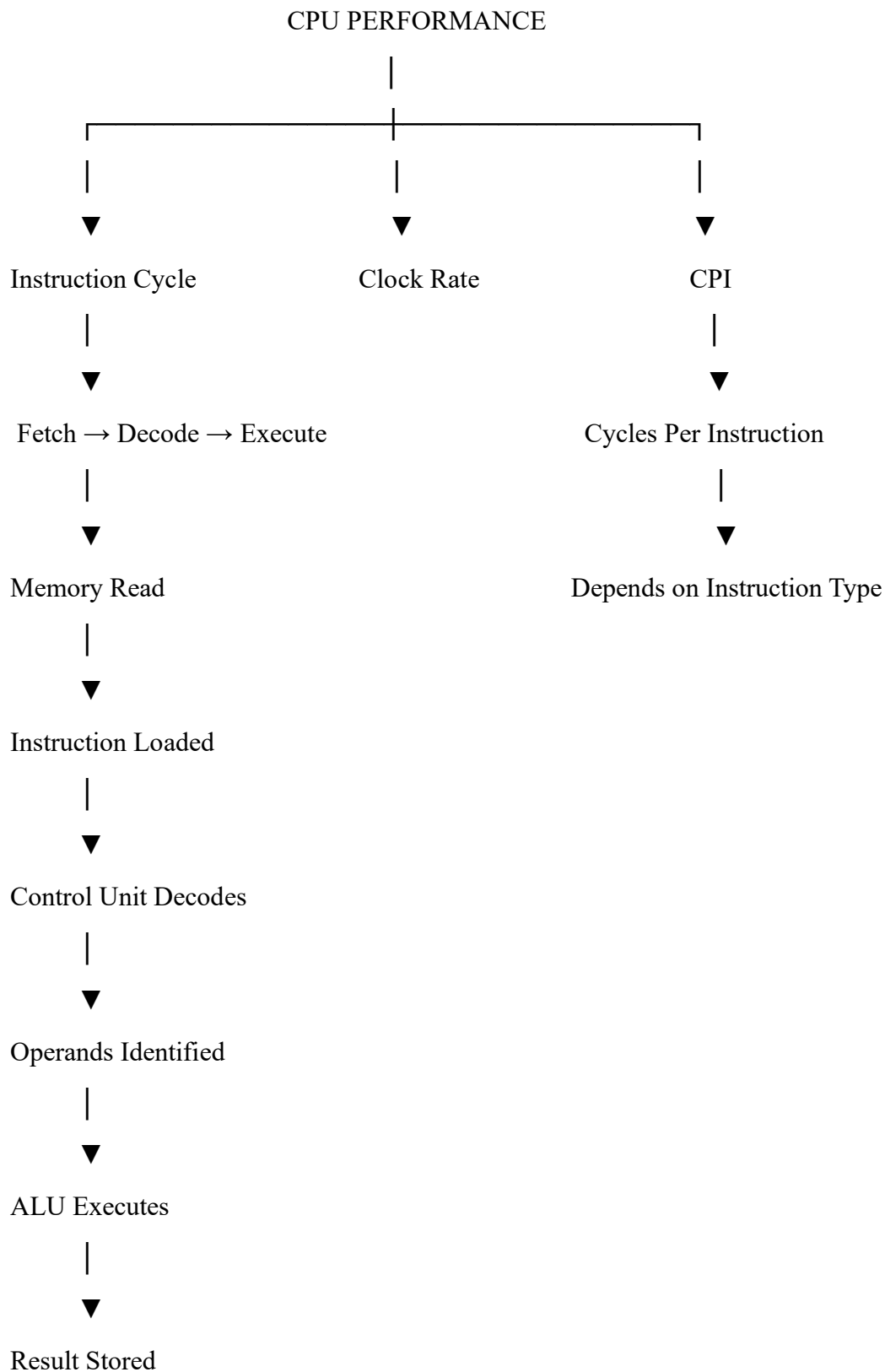
Parallel Data Transfer

Hierarchical Bus



Reduces Traffic and Improves Speed

2. Concept Map: Instruction Execution Cycle and CPU Performance



EXECUTION TIME

Execution Time

|



Instruction Count

×

CPI

×

Clock Cycle Time

Clock Cycle Time

|



1 / Clock Frequency

FACTORS AFFECTING PERFORMANCE

Instruction Complexity

|



Higher CPI

Memory Access Delay

|



Longer Fetch Time

Cache Miss

|



Extra Clock Cycles

Pipeline Stall

|



CPU Waits

Branch Misprediction

|



Pipeline Flush

Interrupts

|



Execution Delay

BENCHMARKING

Benchmark Programs

|



Measure

Execution Time

CPI

Clock Cycles

Throughput

Latency

Examples

SPEC CPU

LINPACK

Geekbench

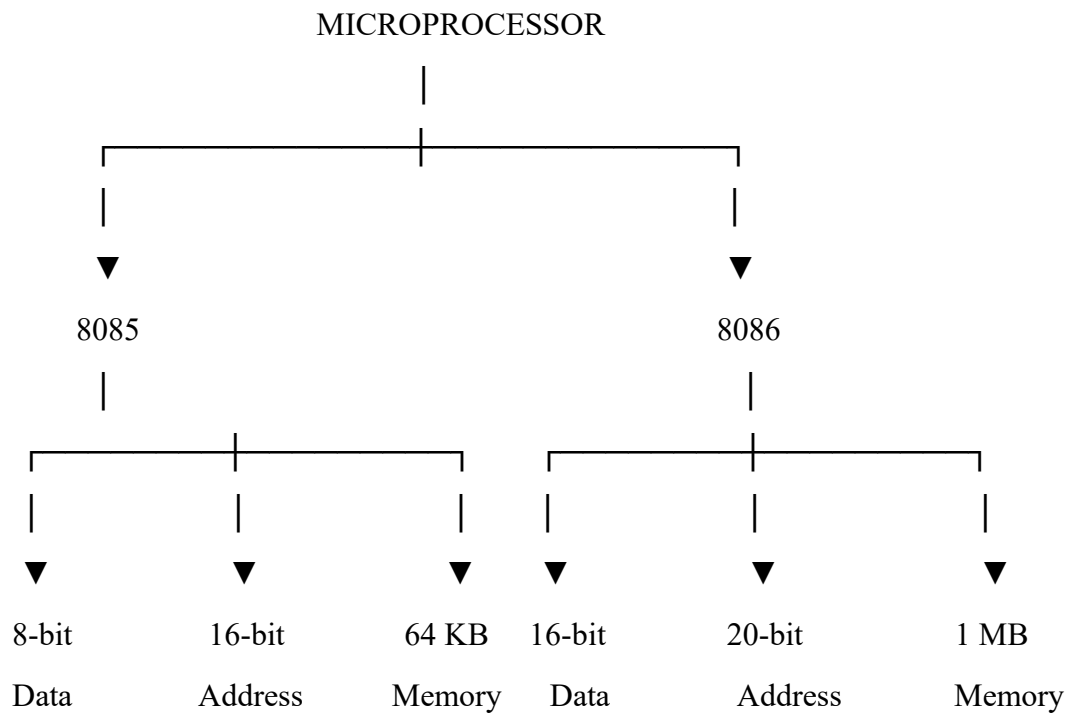
Benchmark Results

|



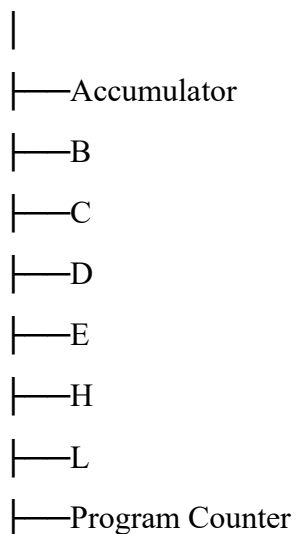
Compare CPU Performance
Optimize Hardware
Evaluate Processor Efficiency

3. Concept Map: Comparison of 8085 and 8086 Architecture



REGISTER ORGANIZATION

8085



└─Stack Pointer

8086

|

└─AX BX CX DX

└─SI DI BP SP

└─CS DS SS ES

└─IP

└─FLAGS

INSTRUCTION SET

8085

|

└─Data Transfer

└─Arithmetic

└─Logical

└─Branch

└─Machine Control

8086

|

└─More Powerful Instructions

└─String Operations

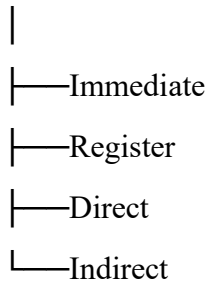
└─Multiplication

└─Division

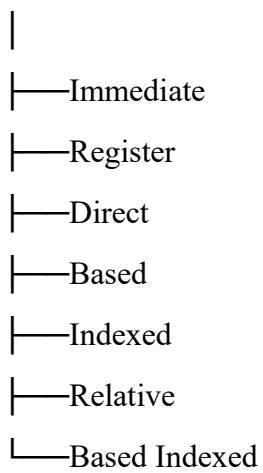
└─Loop Instructions

ADDRESSING MODES

8085



8086



MEMORY SEGMENTATION (8086)

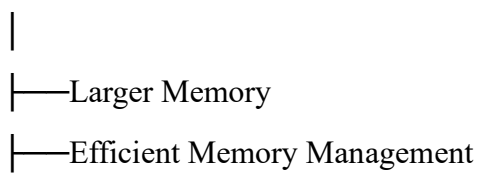
Code Segment

Data Segment

Stack Segment

Extra Segment

Advantages



|—Supports Multitasking

|—Faster Execution

IMPACT ON PERFORMANCE

8085

|

|—Simple Programs

|—Lower Speed

|—Suitable for Basic Embedded Systems

8086

|

|—Higher Speed

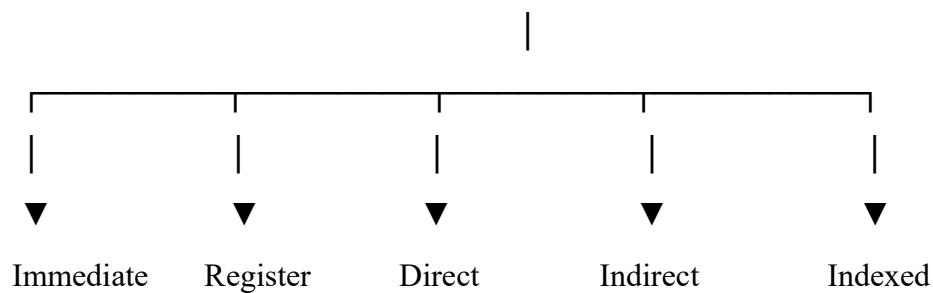
|—More Efficient Execution

|—Supports Large Applications

|—Better Real-Time Performance

4. Concept Map: Addressing Modes and Instruction Types

ADDRESSING MODES



Immediate

|

|—Operand Inside Instruction

|—Fastest

└─No Memory Access

Register

|

└─Operand in Register

└─Very Fast

└─Small Instruction Size

Direct

|

└─Address Given

└─One Memory Access

└─Medium Speed

Indirect

|

└─Address Stored in Register

└─Extra Memory Access

└─Slower

Indexed

|

└─Base + Index

└─Array Processing

└─Efficient Loops

INSTRUCTION TYPES

Data Transfer

|

├—MOV
├—LOAD
└—STORE

Arithmetic

|
├—ADD
├—SUB
├—MUL
└—DIV

Control

|
├—JMP
├—CALL
├—RET
└—LOOP

EFFECT ON PERFORMANCE

Addressing Mode



Memory Access Time



Execution Speed



CPU Performance

Instruction Size

|



Program Memory Usage

REAL WORLD APPLICATIONS

Immediate

|

└─Initialize Variables

Register

|

└─Loop Counters

Direct

|

└─Access Fixed Memory

Indirect

|

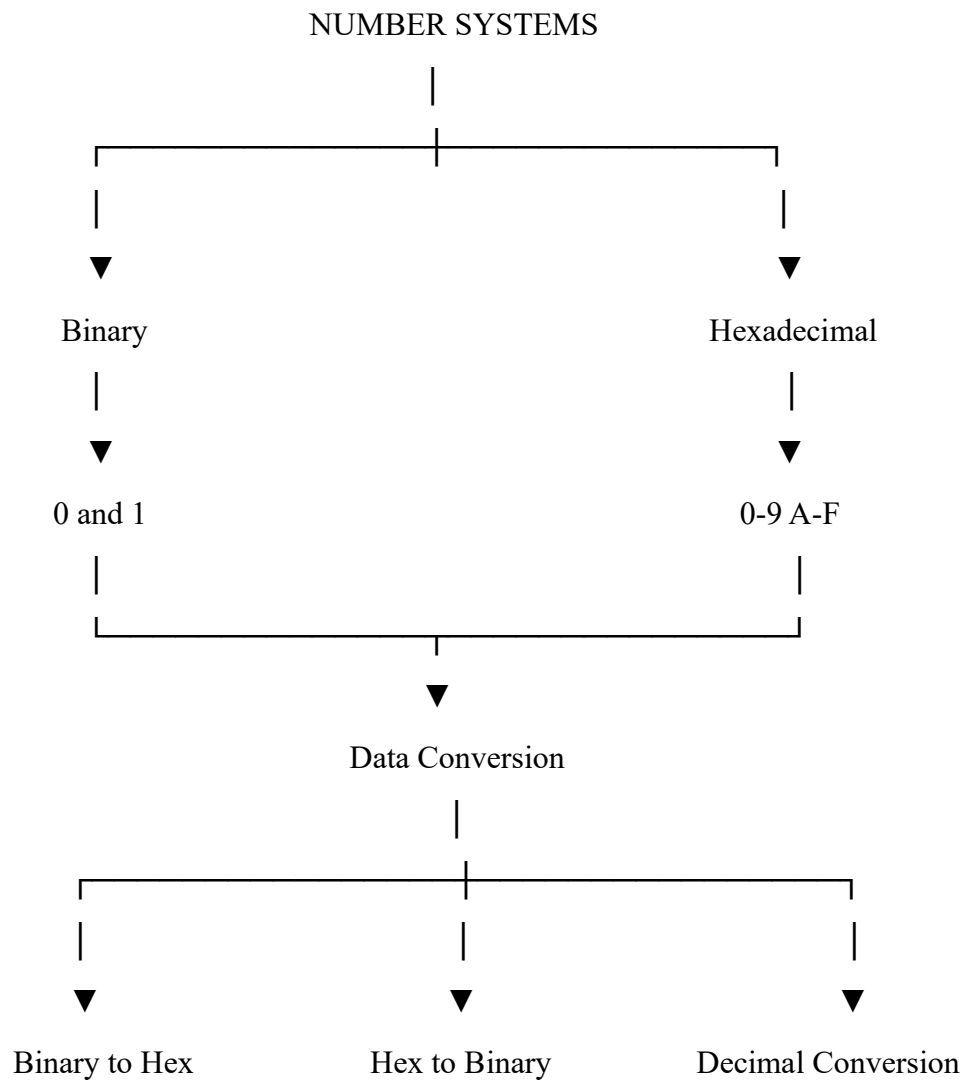
└─Linked Lists

Indexed

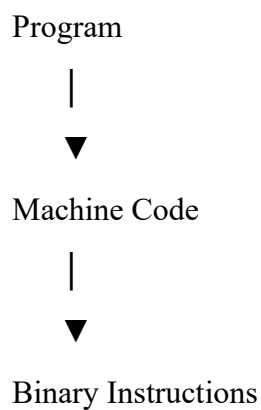
|

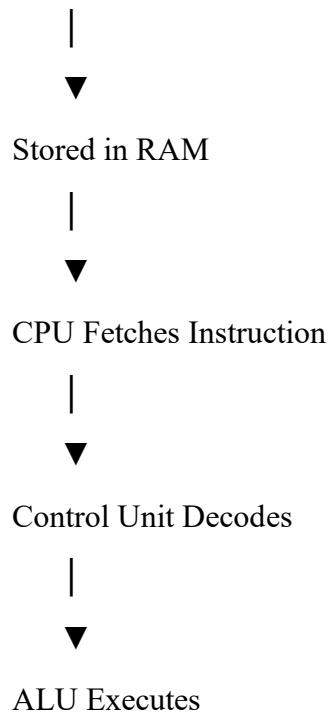
└─Array Processing

5. Concept Map: Number Systems, Memory Storage and Debugging



INSTRUCTION STORAGE





MEMORY ORGANIZATION

RAM

- |
- |—Stores Instructions
- |—Stores Data
- |—Uses Binary

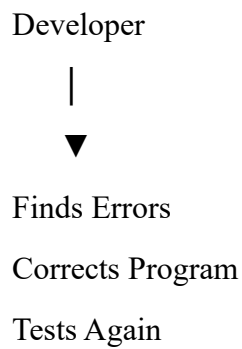
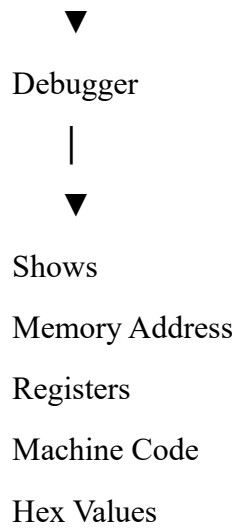
Memory Address

- |
- |—Represented in Hex
- |—Easy to Read

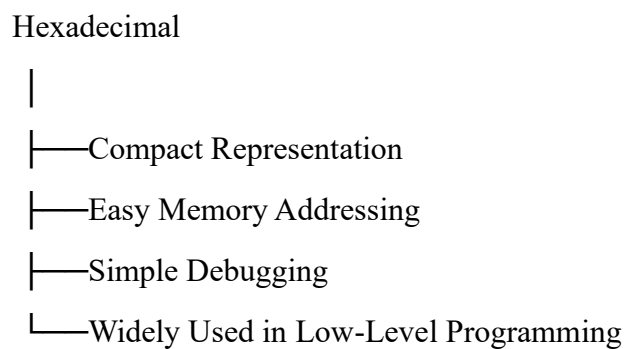
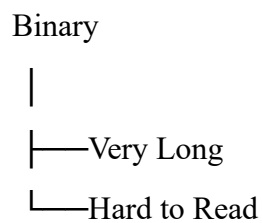
DEBUGGING PROCESS

Program Error

|



WHY HEXADECIMAL IS USED



APPLICATIONS

Binary

|

|—CPU Operations

|—Digital Circuits

|—Memory Storage

Hexadecimal

|

|—Memory Addresses

|—Machine Instructions

|—Debugging Tools

|—Embedded Systems